An Agent Based Approach to High-Fidelity Market Simulation for Interactive Backtesting

Jake Alesi

March 2026

1 Mathematical Framework and Market Physics

1.1 Introduction

Financial markets exhibit complex emergent properties—such as volatility clustering, heavy tails, and liquidity crises—that cannot be adequately captured by standard geometric Brownian motion (GBM). To generate synthetic data that satisfies these "stylized facts" of quantitative finance, we implement a hybrid system.

This section details the top-down physics-based layer responsible for generating the underlying correlated random processes that drive the simulation. Unlike pure Agent-Based Models (ABMs) which rely solely on emergent behavior, our approach utilizes a "God Layer" to enforce fundamental sector correlations and volatility regimes, ensuring the simulation remains grounded in mathematical reality.

1.2 Multivariate Geometric Brownian Motion

We model the fundamental price process of N assets using a Multivariate Geometric Brownian Motion (mGBM). Unlike independent random walks, real-world assets exhibit significant cross-correlation, particularly within industrial sectors.

The price evolution of asset i is governed by the Stochastic Differential Equation (SDE):

$$\frac{dS_t^i}{S_t^i} = \mu_i dt + \sqrt{v_t} dW_t^i \tag{1}$$

Where:

- S_t^i is the price of asset i at time t.
- μ_i is the drift coefficient (expected return).
- v_t is the stochastic variance (volatility squared) at time t.
- W_t^i is a Wiener process correlated across assets.

To induce realistic correlations, we define a covariance matrix $\Sigma \in \mathbb{R}^{N \times N}$. We utilize **Cholesky Decomposition** to decompose Σ into a lower triangular matrix L such that $LL^T = \Sigma$. The correlated random shocks dW_t are generated by transforming a vector of independent standard normal variables $Z_t \sim \mathcal{N}(0, I)$:

$$dW_t = L \cdot Z_t \tag{2}$$

In our implementation, we construct Σ using a **Block-Diagonal** architecture. This simulates distinct market sectors (e.g., Technology, Energy) where intra-sector correlation is high ($\rho \approx 0.65$), while inter-sector correlation remains lower, driven only by a systematic market factor (β).

```python
def _build_correlation_structure(self):
    """Constructs a block-diagonal correlation matrix for realistic sector clustering."""
    block_size = config.N_ASSETS // 4
    blocks = []

    for _ in range(4):
        # Generate internal sector correlation (~0.65) with noise
        base = 0.65
        noise = np.random.uniform(-0.05, 0.05, (block_size, block_size))
        block = np.full((block_size, block_size), base) + noise
        np.fill_diagonal(block, 1.0)
        blocks.append(block)

    # Assemble the block diagonal matrix and add market-wide beta
    matrix = block_diag(*blocks)
    matrix += 0.25

    # Normalize and ensure Positive Semi-Definiteness (PSD)
    d = np.sqrt(np.diag(matrix))
    matrix = matrix / np.outer(d, d)
    eigvals, eigvecs = np.linalg.eigh(matrix)
    eigvals = np.maximum(eigvals, 1e-8)

    return eigvecs @ np.diag(eigvals) @ eigvecs.T
```

Listing 1: Construction of Block-Diagonal Correlation Matrix

1.3 Stochastic Volatility: The Heston Model

A critical flaw in standard GBM simulations is the assumption of constant volatility (σ). In reality, volatility is time-varying, mean-reverting, and clustered. To address this, we implement a discrete-time approximation of the **Heston Model**.

We treat the variance v_t as a stochastic process described by the following SDE:

$$dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t}dW_t^v \tag{3}$$

Where:

- θ is the long-term average variance (e.g., 0.04 for 20% volatility).
- κ is the rate of mean reversion (speed of recovery).
- ξ is the "volatility of volatility," controlling the kurtosis (fat tails).
- dW_t^v is the Wiener process driving the variance.

This model ensures that our simulation exhibits **Volatility Clustering**: periods of high turbulence tend to be followed by further turbulence. The implementation utilizes an Euler-Maruyama discretization scheme to generate a pre-computed volatility path for the simulation duration.

```python
def get_volatility_path(self):
    """
    Simulates the Heston stochastic volatility path.
    """
    # Updated to support Variable Steps (Minute/Second resolution)
    v = np.zeros(config.N_STEPS)
    v[0] = config.INITIAL_VARIANCE
    dt = config.DT

    for t in range(1, config.N_STEPS):
        # Heston SDE discretization
        drift = config.MEAN_REVERSION_SPEED * (config.LONG_TERM_VARIANCE - v[t-1]) * dt

        # The stochastic shock component
        shock = config.VOL_OF_VOL * np.sqrt(max(v[t-1], 0)) * np.sqrt(dt) * np.random.
    normal()
```

```

        # Update variance, ensuring positivity
        v[t] = max(v[t-1] + drift + shock, 1e-6)

    return np.sqrt(v) # Returns Volatility (Sigma)
```

Listing 2: Heston Volatility Path Generation

1.4 Time Discretization and Scaling

The continuous stochastic differential equations (SDEs) are discretized using an Euler-Maruyama scheme with a variable time-step Δt. To support multi-resolution simulation (Daily, Hourly, Minute, Second), the system dynamically scales the drift and diffusion components:

$$\Delta t = \frac{1}{252 \times N_{steps_per_day}} \tag{4}$$

Consequently, the realized volatility over a single step scales with $\sqrt{\Delta t}$, ensuring that the statistical properties of the price process remain consistent regardless of the temporal resolution chosen for the simulation.

2 Market Microstructure and Execution Mechanics

2.1 Introduction

While the Stochastic Physics engine determines the "fair value" (fundamental price) of an asset, real-world transaction prices are determined by the interaction of supply and demand within a Limit Order Book (LOB).

In this implementation, we model the market venue not as a frictionless environment, but as a dynamic system with finite liquidity. This ensures that all participants—whether internal agents or external strategies—face realistic transaction costs via dynamic bid-ask spreads and price impact (slippage).

2.2 Dynamic Bid-Ask Spread Modeling

In a frictionless market, trades occur at the mid-price P_{mid}. However, market makers demand compensation for inventory risk, creating a spread between the best bid B_t and best ask A_t.

We model the spread S_t as a dynamic function of current market volatility σ_t and a discrete panic factor ψ_t. The spread expansion is governed by:

$$S_t = S_{base} \cdot (1 + \alpha\sigma_t) \cdot (1 + \beta\psi_t) \tag{5}$$

Where:

- S_{base} is the minimum baseline spread (e.g., 5 basis points).
- α is the volatility sensitivity coefficient.
- β is the panic multiplier, causing liquidity to evaporate exponentially during crashes.

Consequently, the actionable quotes available to all market participants are:

$$A_t = P_{mid} + \frac{S_t}{2}, \quad B_t = P_{mid} - \frac{S_t}{2} \tag{6}$$

2.3 Market Impact and Liquidity Scaling

To simulate finite liquidity depth, we implement a linear market impact model inspired by Kyle's Lambda (λ). Crucially, we introduce a **Time-Scaling Factor** $\gamma(\Delta t)$ to account for the fact that order book depth is thinner over short time intervals (e.g., 1 second) compared to longer intervals (e.g., 1 day).

The realized execution price P_{exec} for a net order flow Q_t is defined as:

$$P_{exec} = P_{mid} + (\lambda \cdot \gamma(\Delta t)) \left(\frac{Q_t}{L_t} \right) \tag{7}$$

Where L_t represents the available liquidity depth, inversely proportional to the panic factor ψ_t. The scaling factor $\gamma(\Delta t)$ ensures that a large market order exerts a disproportionately larger price impact when executed in a single second compared to being worked over a full trading day.

```python
def execute(self, net_order_flow):
    """
    Matches orders and returns the weighted average execution price.
    Impact: Large orders move the Mid Price permanently.
    """
    # Liquidity Depth decreases as panic increases
    current_liquidity = config.BASE_LIQUIDITY / (1 + panic_factor * 20)

    # Price Impact (Kyle's Lambda)
    # Note: config.PRICE_IMPACT_FACTOR includes the time-scaling gamma(dt)
    market_impact = (net_order_flow / current_liquidity) * (self.mid_price * config.
    PRICE_IMPACT_FACTOR)

    # Update Mid Price based on pressure (Permanent Impact)
    self.mid_price += market_impact

    # Return the 'Last Traded Price'
    # Buyers pay the Ask, Sellers receive the Bid
    if net_order_flow > 0:
        return self.ask
    else:
        return self.bid
```

Listing 3: Execution and Price Impact Logic

3 Interactive Backtesting Framework

3.1 Active vs. Passive Simulation

Standard backtesting methodologies rely on "Passive" simulation, where historical price data is static and unaffected by the strategy being tested. This approach fails to capture **Alpha Decay**: the degradation of strategy performance due to its own market impact.

To address this, our simulator is architected with a **Dual-Mode** design:

1. **Passive Mode (generate_data.py):** Generates synthetic OHLCV data based purely on internal agent interactions. Useful for offline research and initial strategy validation.
2. **Active Mode (main.py):** Injects the User Strategy directly into the simulation loop. The strategy becomes an endogenous participant, capable of moving the market price against itself via the liquidity mechanisms defined in Section 2.

3.2 Order Flow Aggregation

To facilitate active backtesting, the simulation aggregates order flow from both the internal agent population and the external User Strategy. The Net Order Flow Q_t at time t is defined as:

$$Q_t = \sum_{i=1}^{N} q_{agent}^{(i)} + q_{user} \tag{8}$$

Where q_{user} represents the signed volume injected by the strategy being backtested. The realized execution price P_{exec} is then derived via the linear impact model.

This feedback loop implies that if q_{user} is sufficiently large relative to the liquidity depth L_t, it will permanently alter the price trajectory P_{mid}, subsequently influencing the behavior of trend-following agents in future time steps $t + 1$.

3.3 Implementation: The Interactive Execution Loop

For practitioners implementing this simulator, the active workflow requires injecting the strategy logic directly into the main event loop. The execution sequence for a single time step is defined as follows:

1. **Observe:** The strategy receives the current $OrderBook$ state (B_t, A_t) and Volatility Regime (σ_t).
2. **Decide:** The strategy returns a signed order size q_{user}.
3. **Aggregate:** The system sums q_{user} with the noise and trend flow from internal agents.
4. **Execute:** The Order Book matches the net flow, determining the clearing price P_{exec}.

```
# Inside the main simulation loop (main.py):

# 1. OBSERVE
# Strategy views current book state and volatility
# Note: 't' is passed to handle latency/patience logic
user_order = my_strategy.on_data(book, price_history, current_vol, t)

# 2. AGGREGATE
# Combine User flow with Internal Agent flow
net_flow = sum([agent.decide(...) for agent in agents])
net_flow += user_order  # <--- Active Injection

# 3. EXECUTE
# The Order Book clears the total flow
# Large user_order will cause slippage here
close_price = book.execute(net_flow)
```

Listing 4: Active Strategy Injection into Simulation Loop

4 Agent Taxonomy and Behavioral Dynamics

4.1 Heterogeneous Agent Hypothesis

Standard financial models often rely on a "Representative Agent" with rational expectations. However, real markets are driven by the interaction of diverse participants with conflicting goals and time horizons.

Our simulation adopts the **Heterogeneous Agent Hypothesis**, populating the market with four distinct agent classes. The aggregate price process P_t emerges not from a closed-form equation, but from the complex interplay of these agents competing for liquidity in the Order Book.

4.2 Agent Strategy Definitions

We define the decision function D_t for an agent as a mapping from the market history set $H_t = \{P_0, \ldots, P_t\}$ and volatility regime σ_t to a signed order quantity q_t.

4.2.1 Trend Followers (Momentum)

Trend Followers operate on the belief that asset prices exhibit serial correlation. They utilize a Moving Average Crossover strategy to capture momentum.

$$q_t^{trend} = \delta \cdot \text{sgn}(\text{MA}_{short} - \text{MA}_{long}) \tag{9}$$

Where δ is the standard trade size, and sgn is the sign function. This strategy introduces **Positive Feedback Loops** into the system, contributing to bubbles and crashes.

4.2.2 Mean Reverters (Value)

Mean Reverters act as stabilizing forces, betting that prices will return to a long-term equilibrium. They utilize a Bollinger Band logic:

$$q_t^{mean} = \begin{cases} -\delta & \text{if } P_t > \mu_t + k\sigma_{hist} \\ +\delta & \text{if } P_t < \mu_t - k\sigma_{hist} \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

This strategy provides liquidity during small dislocations but can suffer catastrophic losses during regime shifts (e.g., Black Swans).

4.2.3 Institutional Agents (Smart Money/Risk Control)

Institutional agents act as "Volatility Targeters." They accumulate positions slowly to minimize market impact but possess a strict risk constraint. If market volatility σ_t exceeds a critical threshold σ_{crit}, they deleverage aggressively.

$$q_t^{inst} = \begin{cases} -\Gamma \cdot \text{Position}_t & \text{if } \sigma_t > \sigma_{crit} \text{ (Panic De-risking)} \\ \alpha & \text{otherwise (Accumulation)} \end{cases} \tag{11}$$

This behavior mimics the "dash for cash" observed during liquidity crises.

4.2.4 Noise Traders

To ensure continuous liquidity and prevent market stagnation, Noise Traders submit random orders uncorrelated with price history.

$$q_t^{noise} \sim \mathcal{N}(0, \delta) \tag{12}$$

4.3 Agent Latency and Attention Granularity

In high-frequency simulations (e.g., Minute or Second resolution), it is unrealistic for low-frequency agents (such as Trend Followers or Value Investors) to re-evaluate their positions at every time step.

To prevent "Signal Churn" and unrealistic volume explosion, we implement an **Attention Constraint** mechanism governed by a patience parameter k:

$$D_t^{agent} = \begin{cases} \text{Strategy}(H_t) & \text{if } t \equiv 0 \pmod{k} \\ 0 & \text{otherwise} \end{cases} \tag{13}$$

For example, in a 1-second simulation, a Trend Follower may have $k = 60$, implying they only sample the market and trade once per minute, whereas a Market Maker (Noise Trader) may have $k = 1$, interacting continuously. This heterogeneity preserves the fractal nature of market volume across different time scales.

4.4 Emergent Reflexivity

The interaction between these agents creates **Reflexivity**. For example, a random shock (Physics) may trigger a Trend Follower to buy. This buying pressure raises P_t (Microstructure), which confirms the trend, inducing further buying. Simultaneously, Mean Reverters may sell into this strength. The market price is the equilibrium point where the net demand of these opposing forces is zero.

```python
def decide(self, book, history, vol, panic, step_index):
    # --- 1. LATENCY / PATIENCE CHECK ---
    # Agents only trade on their specific time-step modulo
    if step_index % config.AGENT_PATIENCE != 0:
        return 0

    # --- 2. PANIC REGIME SWITCH ---
    if panic > 0.4:
        # Trend followers panic sell; Institutions de-risk
        if self.strategy == 'Trend': return -200
        if self.strategy == 'Institutional': return 0
```

```python

    # --- 3. NORMAL MARKET REGIME ---
    current_price = book.mid_price
    ma_short = np.mean(history[-5:])
    ma_long = np.mean(history[-20:])

    if self.strategy == 'Trend':
        # Momentum Logic
        return 100 if ma_short > ma_long else -100

    elif self.strategy == 'MeanRev':
        # Statistical Arbitrage Logic
        z_score = (current_price - ma_long) / np.std(history[-20:])
        if z_score > 2: return -100  # Overbought
        elif z_score < -2: return 100 # Oversold

    return 0
```

Listing 5: Agent Decision Logic with Latency Check

5 Simulation Configuration and Parameters

5.1 Overview

The simulation is controlled by a centralized configuration module (‘config.py‘) that governs the temporal resolution, physical constants, and agent population dynamics. This modular design allows for switching between low-frequency macro and high-frequency microstructure tests.

5.2 Time Resolution Settings

The simulation engine supports four distinct temporal resolutions. The choice of resolution automatically rescales the stochastic drift ($\mu\Delta t$) and diffusion ($\sigma\sqrt{\Delta t}$) components of the price process.

Mode	Δt	Steps/Day	Use Case
Daily (1D)	1/252	1	Swing Trading / Macro
Hourly (1H)	1/1764	7	Intraday Swing
Minute (1m)	1/98280	390	VWAP / Execution Algo
Second (1s)	$1/5.8M$	23400	HFT / Market Making

Table 1: Temporal Resolution Configurations

5.3 Market Physics Parameters

The underlying asset prices are driven by the Heston Stochastic Volatility model. The following parameters are calibrated to produce realistic "stylized facts" such as fat tails and volatility clustering.

- **Mean Reversion ($\kappa = 4.0$):** The speed at which volatility returns to its long-run average.
- **Long-Term Variance ($\theta = 0.04$):** Corresponds to an annualized volatility of 20%.
- **Vol of Vol ($\xi = 0.7$):** Controls the kurtosis of the distribution (likelihood of extreme events).
- **Correlation Structure:** A block-diagonal matrix representing 4 distinct industrial sectors with $\rho \approx 0.65$ intra-sector correlation.

5.4 Microstructure and Liquidity

To simulate realistic execution costs, the order book parameters scale dynamically with the chosen time resolution.

- **Base Spread:** Fixed at 5 basis points (0.05%).

- **Price Impact Factor (λ):**

$$\lambda_{adjusted} = \lambda_{base} \cdot \gamma(\text{TimeFrame}) \tag{14}$$

where γ increases for higher-frequency settings (e.g., $\gamma_{1s} = 10$) to model the thinner liquidity available in sub-second intervals.

5.5 High-Frequency Safeguards

When operating at micro-resolutions (Minute/Second), specific safeguards are enabled to prevent numerical artifacts:

1. **Black Swan Suppression:** The Panic Regime switch is automatically disabled for $\Delta t = 1s$ to prevent "Flash Crash" artifacts where panic decays faster than human reaction time.
2. **Agent Patience (k):** A latency constraint ensures agents do not over-trade.
   - Daily/Hourly: $k = 1$ (Trade every step)
   - Minute: $k = 5$ (Trade every 5 minutes)
   - Second: $k = 60$ (Trade every minute)

6 Usage Guide and Implementation Strategy

6.1 Strategy Implementation Interface

To interact with the simulator, the user must define their trading logic within the `my_strategy.py` module. The system expects a class named `UserStrategy` exposing a specific public method `on_data`.

The `on_data` method is invoked at every simulation step t (subject to latency constraints) and provides the following state variables:

- `book`**:** The current Order Book object. Access `book.bid`, `book.ask`, and `book.mid_price`.
- `history`**:** A NumPy array of historical closing prices for the asset (H_t).
- `volatility`**:** The current annualized volatility (σ_t) from the Heston model.
- `step_index`**:** The current integer time step t, used for time-dependent logic.

```python
class UserStrategy:
    def __init__(self):
        self.position = 0
        self.cash = 100000.0

    def on_data(self, book, history, volatility, step_index):
        """
        Input:
            book: OrderBook object (book.bid, book.ask)
            history: np.array of past prices
            volatility: float (Heston vol)
            step_index: int (Current time t)
        Output:
            int: Signed order size (Positive = Buy, Negative = Sell)
        """
        # Example: Simple Momentum
        if len(history) < 20: return 0

        ma = np.mean(history[-20:])
        if book.mid_price > ma:
            return 100 # Buy 100 shares
        return 0
```

Listing 6: User Strategy Template (my_strategy.py)

6.2 Execution Modes

The simulation framework provides two distinct entry points depending on the research objective:

6.2.1 Passive Mode (`generate_data.py`)

Use Case: Offline Backtesting / Data Science
Executes the simulation without user intervention. The user strategy is ignored.

- **Input:** `config.py` settings.
- **Process:** Simulates N assets driven by internal agents only.
- **Output:** `phd_market_data.csv` containing OHLCV, Panic Scores, and Order Book snapshots.

6.2.2 Active Mode (`main.py`)

Use Case: Live Strategy Stress-Testing
Executes the simulation with the `UserStrategy` injected into the loop.

- **Input:** Logic defined in `my_strategy.py`.
- **Process:** The user strategy competes for liquidity against internal agents. High order volume will result in slippage (Alpha Decay).
- **Output:** `my_performance.csv` containing a step-by-step log of Cash, Position, Equity, and Execution Prices.

6.3 Configuration Workflow

Before executing either mode, the user must configure the `config.py` file.

1. Select the **Timeframe** (Daily, Hourly, Minute, Second) by uncommenting the relevant block.
2. Verify the **Risk Parameters** (Black Swan probability and severity).
3. Run the desired execution script from the terminal:

```
$ python3 main.py          # For Active Testing
$ python3 generate_data.py # For Data Generation
```